

SİSTEM PROGRAMLAMA LAB 2 - 3 – 4

BİRLEŞTİRİLMİŞ

Video Linkleri:

Lab2:

<https://drive.google.com/file/d/1EchlrEsAydpY-fTrxHAtrS0QUrYSijVJ/view?usp=sharing>

Lab3:

<https://drive.google.com/file/d/1tYK8ZFulmiowH-ph99lQMbXlxJkDzl0D/view?usp=sharing>

Lab4:

<https://drive.google.com/file/d/1d-Nkxdc9eEkRQBSTUSBXpal1jZrKCLsG/view?usp=sharing>

LAB 2 RAPOR

23220030065 Bartu ŞANLI

Bölüm 1 – Child Proseslerin Yaratılması

mainprog.c dosyasında parent process, fork() çağrısıyla üç adet child process oluşturur.

Her child process execl() fonksiyonu kullanarak child adlı programa geçiş yapar.

Parent, wait() sistem çağrısı ile tüm child process'lerin bitmesini bekler ve çıkış durumlarını ekrana yazdırır.

Bu bölümde process yaratma, senkronizasyon ve parent-child ilişkisi gözlemlenmiştir.

```
GNU nano 7.2
/* child.c */
/* mainprog.c için child */
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
#include <stdlib.h>
#include <signal.h>

int main(int argc, char *argv[]) {
    pid_t pid;
    int ret_value;

    pid = getpid();
    ret_value = (int)(pid % 256);

    printf("Child %d status = %02X ile dönecek\n", pid, ret_value);
    srand((unsigned)pid);
    sleep(rand() % 5); /* 0 - 4 s arasında gelişigüzel bir zaman uyu */

    if (atoi(argv[1]) % 2) {
        printf("Child %d SIGNAL 0009 ile sonlanıyor\n", pid);
        kill(pid, 9); /* child kendini öldürüyor */
    } else {
        printf("Child %d exit(%04X) ile sonlanıyor\n", pid, ret_value);
        exit(ret_value);
    }
}
```

```

GNU nano 7.2
/* mainprog.c */
/* Child 'ın başka bir programa geçişi */
/* wait() ve exit() system çağrılarının incelenmesi */
/* child.c ' yide görün */
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
int main ()
{
    pid_t pid,w;
    int i,status;
    char value[3]; /* string */
    for (i=0; i<3;++i)
    {
        /* Parent 3 child process yaratır */
        if ((pid = fork()) ==0)
        {
            sprintf(value, "%d", i); /* değer 0,1 ve 2 olabilir*/
            /* Herbir child geridönüşü olmaksızın child programına geçer */
            execl("child", "child", value, NULL);
        }
        else /* Parent burada devam eder */
        {
            printf("Parent child (ID = %d) yarattı\n", pid);
        }
    }

    /* parent,tüm child ların işini bitirmesini bekler */
    /* w = child ID veya -1 eğer başka bekleyecek child yoksa */
    /*
    while ((w = wait(&status)) && w != -1)
    {
        if (w != -1) /* w = -1, başka child yok */
        printf ("Parent: child ID = %d status = %04X ile döndü\n",
            w, status);
    }
    exit(0);

```

Koddaki Main fonksiyonu öncesi int kullanımının unutulması gibi basit hatalar düzeltilmiştir.

```
bartu_sanli@bartu-computer:~/lab2$ ./mainprog
Parent child (ID = 692) yarattı
Parent child (ID = 693) yarattı
Parent child (ID = 694) yarattı
Child 692 status = B4 ile dönecek
Child 693 status = B5 ile dönecek
Child 694 status = B6 ile dönecek
Child 693 SIGNAL 0009 ile sonlanıyor
Parent: child ID = 693 status = 0009 ile döndü
Child 692 exit(00B4) ile sonlanıyor
Parent: child ID = 692 status = B400 ile döndü
Child 694 exit(00B6) ile sonlanıyor
Parent: child ID = 694 status = B600 ile döndü
```

Kodun çalışmış hali Ubuntu ortamında böyle çıktı vermektedir

Bölüm 2 – Child Prosesin Başka Bir Programa Geçişi

procident.c dosyasında fork() ile bir child process oluşturulur.

Parent process kısa sürede sonlanırken, child process daha uzun süre çalışmaya devam eder.

Parent ölünce child yetim hale gelir ve init sürecine (PID 1) bağlanır.

Ardından execl() çağrısıyla child process, simple adlı yeni programa dönüşür.

Bu bölümde orphan process oluşumu ve execl() ile program dönüşümü incelenmiştir.

GNU nano 7.2

```
/* child.c */
/* mainprog.c için child */
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
#include <stdlib.h>
#include <signal.h>

int main(int argc, char *argv[]) {
    pid_t pid;
    int ret_value;

    pid = getpid();
    ret_value = (int)(pid % 256);

    printf("Child %d status = %02X ile dönecek\n", pid, ret_value);
    srand((unsigned)pid);
    sleep(rand() % 5); /* 0 - 4 s arasında gelişigüzel bir zaman uyu */

    if (atoi(argv[1]) % 2) {
        printf("Child %d SIGNAL 0009 ile sonlanıyor\n", pid);
        kill(pid, 9); /* child kendini öldürüyor */
    } else {
        printf("Child %d exit(%04X) ile sonlanıyor\n", pid, ret_value);
        exit(ret_value);
    }
}
```

```

GNU nano 7.2
/* procident.c */
/* Process ID (PID) çalışması */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
    int i = 0;
    pid_t pid, parent, child, myparent;

    pid = fork(); /* Parent bir child process yaratır */

    if (pid == 0) { /* sadece child için */
        child = getpid();
        i = 2;
        printf("Child: my ID = %d, i = %d\n", child, i);
        myparent = getppid();
        printf("Child: my parent ID = %d\n", myparent);
        sleep(3); /* Child 10 saniye uyur */
        child = getpid();
        printf("\nChild uyuduktan sonra: my ID = %d\n", child);
        myparent = getppid();
        printf("Child uyuduktan sonra: my parent ID = %d\n", myparent);
        execl("./simple", "simple", NULL); /* simple programına geçiş */
        perror("execl failed");
        exit(1);
    }
    else {
        sleep(10);
        parent = getpid();
        printf("Parent: my ID = %d, i = %d\n", parent, i);
        printf("Parent: sonlanıyor...\n");
    }
}

```

Koddaki belli başlı hatalar düzeltilmiştir ve deneme amaçlı parent ve child daki 3 ve 10 parametreleri yer değiştirilmiştir. Bunun sonucunda önce parent değil child ölmüş olur.

```
bartu_sanli@bartu-computer:~/lab2$ ./procident
Child: my ID = 698, i = 2
Child: my parent ID = 697

Child uyuduktan sonra: my ID = 698
Child uyuduktan sonra: my parent ID = 697
YENİ PROGRAM simple CHILD PROCESSCE BAŞLATILDI
Child: my ID = 698
Child: my parent ID = 697
Child: sonlanıyor...
Parent: my ID = 697, i = 0
Parent: sonlanıyor...
```

Kod çalıştığında ise böyle bir çıktı verir.

Bölüm 3 – Proses Bellek Yapısının Anlaşılması

procmemory.c programı, process'in **text**, **data**, **bss**, **heap** ve **stack** alanlarının adreslerini ekrana yazdırır.

malloc() fonksiyonu ile heap bölgesinden dinamik bellek ayrılır, otomatik değişkenler stack üzerinde tutulur.

fork() eklendiğinde her iki process aynı sanal adresleri paylaşır, ancak fiziksel bellekleri birbirinden bağımsızdır.

Bu bölümde Unix'in sanal bellek yapısı ve Copy-On-Write mekanizması gözlemlenmiştir.

```

GNU nano 7.2                                     procmemory.c
/* procmemory.c */
/* Program to display address information about the process */

#include <stdio.h>
#include <string.h>
#include <sys/types.h>
#include <stdlib.h>
#include <unistd.h>

#define SHW_ADR(ID, I) (printf("ID %s \t is at virtual address: %p\n", ID, (void *)&I))

extern int etext, edata, end; /* Harici deęişkenler */

char *cptr = "Bu mesaj showit() fonk. tarafından gösterilir\n";
char buffer1[25];

int showit(char *p);

int main() {
    int i = 0; /* otomatik deęişken */

    fork();
    printf("\nAddress etext: %p\n", &etext);
    printf("Address edata: %p\n", &edata);
    printf("Address end : %p\n", &end);

    SHW_ADR("main", main);
    SHW_ADR("showit", showit);
    SHW_ADR("cptr", cptr);
    SHW_ADR("buffer1", buffer1);
    SHW_ADR("i", i);

    strcpy(buffer1, "A demonstration\n");
    write(1, buffer1, strlen(buffer1) + 1);
    showit(cptr);

    return 0;
}

```

Koddaki hataların düzeltilmiş hali bu şekildedir. Deney amaçlı `int i=0` kısmının altına `fork()` eklenerek child parent oluşturuluş ve birnevi kod 2 kere çalışır gibi bir sonuç vermiştir.


```
Address etext: 0x57c0ec947529
Address edata: 0x57c0ec94a018
Address end   : 0x57c0ec94a050
ID main       is at virtual address: 0x57c0ec947269
ID showit     is at virtual address: 0x57c0ec94742c
ID cptr       is at virtual address: 0x57c0ec94a010
ID buffer1    is at virtual address: 0x57c0ec94a030

ID i          is at virtual address: 0x7ffd2287dc44
Address etext: 0x57c0ec947529
Address edata: 0x57c0ec94a018
Address end   : 0x57c0ec94a050
A demonstration
ID main       is at virtual address: 0x57c0ec947269
ID buffer2    is at virtual address: 0x7ffd2287dc20
ID showit     is at virtual address: 0x57c0ec94742c
Bellek 0x57c10bef46b0 de rezerve edildi
ID cptr       is at virtual address: 0x57c0ec94a010
Bu mesaj showit() fonk. tarafından gösterilir
ID buffer1    is at virtual address: 0x57c0ec94a030
ID i          is at virtual address: 0x7ffd2287dc44
A demonstration
ID buffer2    is at virtual address: 0x7ffd2287dc20
Bellek 0x57c10bef46b0 de rezerve edildi
Bu mesaj showit() fonk. tarafından gösterilir
```

Kod çalıştırıldığında ise bu şekilde sonuç vermektedir.

<https://drive.google.com/file/d/1EchlEsAydP-Y-fTrxHAtrS0QUrYSijVJ/view?usp=sharing>

LAB 3 RAPOR

23220030065 Bartu Şanlı

<https://drive.google.com/file/d/1tYK8ZFulmiowH-ph99lQMbXlXJkDzl0D/view?usp=sharing>

Bölüm-1 : Yeni bir program yazma

```
GNU nano 7.2                                     copy_para1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/stat.h>

#define BLOCK_SIZE 10    // okuma/yazma bloğu boyutu
#define N 10000          // busy waiting döngü sayısı (deney için değiştirilecek)

int main(int argc, char *argv[]) {
    int fd1, fd2, fd3;
    pid_t pid;
    char buffer[BLOCK_SIZE];
    ssize_t bytes_read;

    if (argc != 4) {
        fprintf(stderr, "Kullanım: %s dosya1 dosya2 dosya3\n", argv[0]);
        exit(1);
    }

    // Dosya1'i okumak için aç
    fd1 = open(argv[1], O_RDONLY);
    if (fd1 < 0) {
        perror("dosya1 açılmadı");
        exit(1);
    }

    // Child process oluştur
    pid = fork();

    if (pid < 0) {
        perror("fork hatası");
        exit(1);
    }
}
```

```
// CHILD PROCESS
else if (pid == 0) {
    fd3 = open(argv[3], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd3 < 0) {
        perror("dosya3 oluşturulamadı");
        exit(1);
    }

    // dosya1'den okuyup dosya3'e yaz
    while ((bytes_read = read(fd1, buffer, BLOCK_SIZE)) > 0) {
        for (int i = 0; i < N; i++); // Busy waiting
        write(fd3, buffer, bytes_read);
    }

    close(fd3);
    close(fd1);
    exit(0);
}

// PARENT PROCESS
else {
    fd2 = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd2 < 0) {
        perror("dosya2 oluşturulamadı");
        exit(1);
    }

    // dosya1'den okuyup dosya2'ye yaz
    while ((bytes_read = read(fd1, buffer, BLOCK_SIZE)) > 0) {
        for (int i = 0; i < N; i++); // Busy waiting
        write(fd2, buffer, bytes_read);
    }

    close(fd2);
    close(fd1);
}
```

```
    close(fd2);
    close(fd1);
}

return 0;
}
```

1. Bölüm için yazılan kodlar bu şekildedir.

Bu bölümde, fork kullanarak aynı anda çalışan iki processin (parent ve child) aynı dosya üzerinde işlem yapması incelenmektedir.

Parent process ve child process, aynı giriş dosyasını (dosya1) **bloklar halinde okuyup** sırasıyla dosya2 ve dosya3 isimli iki farklı dosyaya kopyalama işlemi yapmaktadır.

Programın davranışı, N değerine göre değişmiştir.

N değeri arttıkça dosyalar arasındaki fark daha da fazla olmaktadır.

Kod dosya 1 isimli dosyayı aynı anda parent ile dosya2 child ile dosya3'e kopyalamaya çalıştığı için dosyada dozulmalar görülmektedir.

```
bartu_sanli@bartu-computer:~/lab3$ diff dosya1.txt dosya2.txt
1,5c1,3
< Merhaba, bu bir test dosyasıdır. Ve daha başarılı sonuç verebimesi
< için bu text yazısını olabildigince uzatmaya calisiyorum. dsfafhdl
< dsahgfsagdfhdsagfsdaljfgsljdgfljdasggasdfj
< dsfahgdjkkfasggjgfskhdgfsdhgfhkdsagfjksgdkfjasgdkjf
< Bu rastgele karakterler test amaclidir.
---
> Merhaba, bu bir test dosyasıdır. Ve daha başarılı sonu❖si
> için bu text yazısını ole uzatmaya calisiyordl
> dsahgfssdaljfgsljdgfljdasgghgdjkkfasggjdghgfhkdsagfjksgdkfjasastgele katest am
> aclidir.

bartu_sanli@bartu-computer:~/lab3$ diff dosya1.txt dosya3.txt
1,5c1,3
< Merhaba, bu bir test dosyasıdır. Ve daha başarılı sonuç verebimesi
< için bu text yazısını olabildigince uzatmaya calisiyorum. dsfafhdl
< dsahgfsagdfhdsagfsdaljfgsljdgfljdasggasdfj
< dsfahgdjkkfasggjgfskhdgfsdhgfhkdsagfjksgdkfjasgdkjf
< Bu rastgele karakterler test amaclidir.
---
> ❖ verebimeabilidigincum. dsfafhagdfhdsagfasdfj
> dsfagfskhdgfsdhgdkjf
> Bu rrakterler
\ No newline at end of file
```

Çıktılardan da anlaşılacağı gibi dosya2 + dosya 3 = dosya 1 durumu oluşmaktadır.

Bunun sebebi dosya1 dosyasının okunarak kopyalanması ve hem child hem de parent process indexin olduğu yerden içeriği kopyalamasıdır.

Bu offset sorunu olarak adlandırılabilir.

3.Programı birçok kez aynı parametrelerle çalıştırınız. Dosya1, Dosya2 ve Dosya3 birbirlerinin kopyası oldu mu? Sonuçları açıklayınız.

Dolayısıyla bu sorunun cevabı yukarıda anlatılan sebeplerle hayır olmaktadır.

Bölüm-2 : Bölüm-1 deki programın güncellenmesi

2. Bölümde amacımız 1. Bölümdeki kodun güncellenerek offset probleminin ortadan kaldırılmasını ve bunun sayesinde dosya1=dosya2=dosya3 şeklinde dosya1'in fork() aracılığıyla parent ve child kullanarak dosya2 ve dosya3'e başarıyla kopyalanmasını sağlamaktır.

Bu işlem koddan Busy Waiting kısımlarının kaldırılması sayesinde olmuştur.

Yeni güncel kod şu şekildedir:

```
GNU nano 7.2                                copy_parallel_yeni.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/stat.h>

#define BLOCK_SIZE 10
#define N 10000

int main(int argc, char *argv[]) {
    int fd1, fd2, fd3;
    pid_t pid;
    char buffer[BLOCK_SIZE];
    ssize_t bytes_read;

    if (argc != 4) {
        fprintf(stderr, "Kullanım: %s dosya1 dosya2 dosya3\n", argv[0]);
        exit(1);
    }

    // dosya1'i okumak için aç
    fd1 = open(argv[1], O_RDONLY);
    if (fd1 < 0) {
        perror("dosya1 açılmadı");
        exit(1);
    }

    pid = fork();
    if (pid < 0) {
        perror("fork hatası");
        exit(1);
    }

    // ÇUŞTİP
```

```
GNU nano 7.2                                copy_parallel_yeni.c

// CHILD
else if (pid == 0) {
    fd3 = open(argv[3], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd3 < 0) {
        perror("dosya3 oluşturulamadı");
        exit(1);
    }

    while ((bytes_read = read(fd1, buffer, BLOCK_SIZE)) > 0) {
        write(fd3, buffer, bytes_read);
    }

    close(fd3);
    close(fd1);
    exit(0);
}

// PARENT
else {
    fd2 = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd2 < 0) {
        perror("dosya2 oluşturulamadı");
        exit(1);
    }

    while ((bytes_read = read(fd1, buffer, BLOCK_SIZE)) > 0) {
        write(fd2, buffer, bytes_read);
    }

    close(fd2);
    close(fd1);
}

return 0;
}
```

Altteki çıktılarda ise 2. Bölümün amacına ulaştığını görmekteyiz dosya1 dosya2 ve dosya3 tamamen aynı içeriğe sahip olmuştur.

```
GNU nano 7.2                                dosya1.txt
Merhaba, bu bir test dosyasıdır. Ve daha başarılı sonuç verebimesi
icin bu text yazısını olabildigince uzatmaya calisiyorum. dsfafhdl
dsahgfsagdfhdsagfsdaljfgsljdgfljdasggasdfj
dsfahgdjkgfasgjgfskhdgfshdgfjkdsagfjksgdkfjasgdkjf
Bu rastgele karakterler test amaclidir.
```

GNU nano 7.2

dosya2.txt

Merhaba, bu bir test dosyasıdır. Ve daha başarılı sonuç verebimesi için bu text yazısını olabildigince uzatmaya calisiyorum. dsfafhdl dsahgfsagdfhdsagfsdaljfgsljdgfljdasggasdfj dsfahgdjckfasgjgfskhdgfs hdgfkdsagfjksgdkfjasgdkjf Bu rastgele karakterler test amaclidir.

GNU nano 7.2

dosya3.txt

Merhaba, bu bir test dosyasıdır. Ve daha başarılı sonuç verebimesi için bu text yazısını olabildigince uzatmaya calisiyorum. dsfafhdl dsahgfsagdfhdsagfsdaljfgsljdgfljdasggasdfj dsfahgdjckfasgjgfskhdgfs hdgfkdsagfjksgdkfjasgdkjf Bu rastgele karakterler test amaclidir.

Bölüm-3 : Sistem çağrıları fork, exec, wait, sleep, system detaylı incelenmesi

Bu program fork() ile bir child oluşturur.

child, ekrana **"Today's date is:"** yazarken, parent process **execv()** çağrısı ile **sistemin date komutunu** çalıştırarak **tarih ve saati** ekrana basar.

Yani özetle:

Program, iki ayrı process kullanarak önce bir metin mesajı, ardından sistem tarihini ekrana yazdırır.

```
GNU nano 7.2                               inceleme.c
#include <sys/types.h>
#include <stdlib.h>
#include <unistd.h>
#include <stdio.h>

int main(void) {
    pid_t pid;
    char *args[4];

    if ((pid = fork()) < 0) {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0) {
        execl("/bin/echo", "echo", "Today's", "date", "is:", NULL);

        /*
        * eğer exec başarılı ise buraya gelmeyecektir.
        */
        perror("exec failed in child");
        exit(1);
    }
}
```



```
/*
 * Bu kodu sadece parent işletecek
 */
/* burada execl için parametreleri hazırlıyor ..execl exec ailesinin başka v
args[0] = "date";
args[1] = "+Date = %D Time = %H:%M";
args[2] = NULL;

execl("/bin/date", args);
/*
 * eğer exec başarılı ise buraya gelmeyecektir..
 */
perror("exec failed in parent");
exit(1);
}
|
```

Hatalı kod stdlib.h ve stdio.h kütüphaneleri eklenerek ve 0 değeri NULL ile değiştirilerek giderilmiştir.

```
bartu_sanli@bartu-computer:~/lab3$ ./inceleme
Date = 10/23/25 Time = 23:05
bartu_sanli@bartu-computer:~/lab3$ Today's date is:
./inceleme
Date = 10/23/25 Time = 23:05
Today's date is:
bartu_sanli@bartu-computer:~/lab3$ ./inceleme
Date = 10/23/25 Time = 23:05
Today's date is:
bartu_sanli@bartu-computer:~/lab3$ ./inceleme
Date = 10/23/25 Time = 23:05
bartu_sanli@bartu-computer:~/lab3$ Today's date is:
./inceleme
Date = 10/23/25 Time = 23:05
Today's date is:
bartu_sanli@bartu-computer:~/lab3$ ./inceleme
Date = 10/23/25 Time = 23:05
Today's date is:
bartu_sanli@bartu-computer:~/lab3$ ./inceleme
Date = 10/23/25 Time = 23:06
Today's date is:
bartu_sanli@bartu-computer:~/lab3$ ./inceleme
Today's date is:
Date = 10/23/25 Time = 23:06
bartu_sanli@bartu-computer:~/lab3$ |
```

Programın bir çok kez çalıştırılması sonucu parent ve child'ın belli bir sırada çalışmadığı gözlemlenmiştir.

Wait komutu kullanılarak bu durum ortadan kaldırılacaktır.

```
#include <sys/wait.h>
#include <sys/types.h>
#include <stdlib.h>
#include <unistd.h>
#include <stdio.h>

int main(void) {
    pid_t pid;
    char *args[4];

    if ((pid = fork()) < 0) {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0) {
        execl("/bin/echo", "echo", "Today's", "date", "is:", NULL);

        /*
        * eğer exec başarılı ise buraya gelmeyecektir.
        */

        perror("exec failed in child");
        exit(1);
    }
}
```

```
GNU nano 7.2                               inceleme_yeni.c

exit(1);
}
wait(NULL);
/*
 * Bu kodu sadece parent işletecek
 */
/* burada execv için parametreleri hazırlıyor ..execv exec ailesinin başka >
 */
args[0] = "date";
args[1] = "+Date = %D Time = %H:%M";
args[2] = NULL;

execv("/bin/date", args);
/*
 * eğer exec başarılı ise buraya gelmeyecektir..
 */
perror("exec failed in parent");
exit(1);
}
|
```

Eklenen tek bir Wait() komutu sayesinde artık parent ve child sıralı bir şekilde çalışmaktadır.

Önce Child process'in çalışması sayesinde ise çıktı bu hale dönüşmüştür.

```
bartu_sanli@bartu-computer:~/lab3$ ./inceleme_yeni
Today's date is:
Date = 10/23/25 Time = 23:16
bartu_sanli@bartu-computer:~/lab3$ ./inceleme_yeni
Today's date is:
Date = 10/23/25 Time = 23:16
bartu_sanli@bartu-computer:~/lab3$ ./inceleme_yeni
Today's date is:
Date = 10/23/25 Time = 23:16
bartu_sanli@bartu-computer:~/lab3$ ./inceleme_yeni
Today's date is:
Date = 10/23/25 Time = 23:16
bartu_sanli@bartu-computer:~/lab3$ ./inceleme_yeni
Today's date is:
Date = 10/23/25 Time = 23:16
bartu_sanli@bartu-computer:~/lab3$ ./inceleme_yeni
Today's date is:
Date = 10/23/25 Time = 23:16
bartu_sanli@bartu-computer:~/lab3$ ./inceleme_yeni
Today's date is:
Date = 10/23/25 Time = 23:16
bartu_sanli@bartu-computer:~/lab3$ ./inceleme_yeni
Today's date is:
Date = 10/23/25 Time = 23:16
```

Her seferinde istenilen sonucu vermektedir.

Bir sonraki aşama için bizden `execl()` yerine `system()` koymamızı istemektedir.

Bunu `inceleme_yeni.c` üzerinden yaptığımızda ise

```
system("echo Todays date is:");
```

```
bartu_sanli@bartu-computer:~/lab3$ ./inceleme_yeni
Todays date is:
Date = 10/23/25 Time = 23:27
```

Çıktı yine bu şekilde olmaktadır.

Execv() ve execl() arasında ise çok basit farklar vardır.

Execl()'de argümanlar tek tek verilirken

Örneğin : execl("/bin/echo","echo","hi",NULL);

execv()'de argümanlar dizi olarak verilir.

Örneğin : char *a[]={ "echo","hi",NULL}; execv("/bin/echo",a);

Execl() ve System() farkı ise kısaca Execl() tamamen yeni bir process oluştururken System() ise sadece bir powershell komutu yazar mevcut process devam eder.

LAB 4 RAPOR

SİSTEM PROGRAMLAMA

Deney #4 : İş-Parçacıklarını (Thread)

Anlama

Hazırlayan: Bartu Şanlı

Öğrenci No: 23220030065

Açıklayıcı Video:

<https://drive.google.com/file/d/1d-Nkxdc9eEkRQBSTUSBXpal1jZrKCLsG/view?usp=sharing>

1. Deneyin Amacı

Bu deneyin amacı, thread (iş parçacığı) kavramını ve thread tabanlı programlamayı öğrenmektir. Ayrıca, tek-threadli ve çok-threadli programların çalışma mantığını karşılaştırmak, pthread kütüphanesinin kullanımını öğrenmek ve thread'ler ile process'ler arasındaki farkları gözlemlemektir.

2. Deney Adımları ve Sonuçları

Ön Hazırlık: func1 ve func2 fonksionları yazım hataları düzeltilerek ve paket açma kodu eklenerek derlenmiştir.

```
GNU nano 7.2                                func1.c *
/* func1.c */
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>

void func1(void)
{
    int i;
    for (i = 1; i < 10; ++i)
    {
        printf("Fonksiyon func1 yazar ve 4 saniye uyur: %d/10 \n", i);
        sleep(4);
    }
    return;
}
//Paket açma
void *func1_thread(void *arg)
{
    (void)arg;
    func1();                /* aynı işi yapar */
    return NULL;
}
```

```
GNU nano 7.2                                func2.c *
/* func2.c */
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>

void func2(void)
{
    int i;
    for (i = 1; i < 8; ++i)
    {
        printf("Fonksiyon func2 yazar ve 3 saniye uyur: %d/8 \n", i);
        sleep(3);
    }
    return;
}
//paket açma
}
void *func2_thread(void *arg)
{
    (void)arg;
    func2();
    return NULL;
}
```

Adım 1 – singlethread.c: Tek-threadli program derlenip çalıştırılmıştır. Program ana thread üzerinden func1 ve func2 fonksiyonlarını sırasıyla çağırır. Toplam çalışma süresi yaklaşık 69 saniye ölçülmüştür.

Singlethread.c kodu bu şekilde iken:


```

GNU nano 7.2                                singlethread.c
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
#include <sys/time.h>
#include <pthread.h>

/* Fonk. Tanımları */
void func1(void);
void func2(void);

int r1 = 0, r2 = 0; /* Global variables, gerekli olduğunda fonksiyonlar kul>

int
main(void)
{
    pthread_t td1, td2; /* 2 child thread için tanımlayıcılar*/
    int p; /* burada kullanılmıyor */
    int j;
    float delta;
    struct timeval time1, time2;

    printf("Tek-Threadli process başlıyor...\n");
    gettimeofday(&time1, 0); /* Başlama zamanı */

    /* 2 child thread yaratılması */
    p = pthread_create(&td1, NULL, (void *)func1, NULL);

    if (p != 0) {perror("Thread 1 creation problem"); exit(1);}

    p = pthread_create(&td2, NULL, (void *)func2, NULL);

    if (p != 0) {perror("Thread 2 creation problem"); exit(1);}

    /* şimdi tek process, tek ana threadi ile bir şeyler yapıyor */
    for (j=1; j<=4; ++j)
    {
        printf("Processin ana fonksiyonu: %d/4\n", j);

        sleep(3); /* 3 saniye uyu */
    }

    /* Tüm processler sıra ile iki fonksiyonu çağırıyorlar */
    func1();
    func2();

    /* Processin toplam çalışma zamanlarını hesaplama */
    gettimeofday (&time2, 0);
    delta = (float)((1000000*time2.tv_sec + time2.tv_usec) -
                    (1000000*time1.tv_sec + time1.tv_usec))/1000000;
    printf("Toplam geçen zaman = %f saniye\n", delta);

    printf("Ana thread sonlandı...\n");

    return 0;
}

```

Singlethread.c çıktısı ise bu şekilde gözlemlenmiştir:

```
bartu_sanli@bartu-computer:~/Lab3a$ ./singlethread
Tek-Threadli process başlıyor...
Fonksiyon func1 yazar ve 4 saniye uyur: 1/10
Processin ana fonksiyonu: 1/4
Fonksiyon func2 yazar ve 3 saniye uyur: 1/8
Processin ana fonksiyonu: 2/4
Fonksiyon func2 yazar ve 3 saniye uyur: 2/8
Fonksiyon func1 yazar ve 4 saniye uyur: 2/10
Processin ana fonksiyonu: 3/4
Fonksiyon func2 yazar ve 3 saniye uyur: 3/8
Fonksiyon func1 yazar ve 4 saniye uyur: 3/10
Processin ana fonksiyonu: 4/4
Fonksiyon func2 yazar ve 3 saniye uyur: 4/8
Fonksiyon func1 yazar ve 4 saniye uyur: 4/10
Fonksiyon func1 yazar ve 4 saniye uyur: 1/10
Fonksiyon func2 yazar ve 3 saniye uyur: 5/8
Fonksiyon func2 yazar ve 3 saniye uyur: 6/8
Fonksiyon func1 yazar ve 4 saniye uyur: 5/10
Fonksiyon func1 yazar ve 4 saniye uyur: 2/10
Fonksiyon func2 yazar ve 3 saniye uyur: 7/8
Fonksiyon func1 yazar ve 4 saniye uyur: 3/10
Fonksiyon func1 yazar ve 4 saniye uyur: 6/10
Fonksiyon func1 yazar ve 4 saniye uyur: 4/10
Fonksiyon func1 yazar ve 4 saniye uyur: 7/10
Fonksiyon func1 yazar ve 4 saniye uyur: 5/10
Fonksiyon func1 yazar ve 4 saniye uyur: 8/10
Fonksiyon func1 yazar ve 4 saniye uyur: 6/10
Fonksiyon func1 yazar ve 4 saniye uyur: 9/10
Fonksiyon func1 yazar ve 4 saniye uyur: 7/10
Fonksiyon func1 yazar ve 4 saniye uyur: 8/10
Fonksiyon func1 yazar ve 4 saniye uyur: 9/10
Fonksiyon func2 yazar ve 3 saniye uyur: 1/8
Fonksiyon func2 yazar ve 3 saniye uyur: 2/8
Fonksiyon func2 yazar ve 3 saniye uyur: 3/8
Fonksiyon func2 yazar ve 3 saniye uyur: 4/8
Fonksiyon func2 yazar ve 3 saniye uyur: 5/8
Fonksiyon func2 yazar ve 3 saniye uyur: 6/8
Fonksiyon func2 yazar ve 3 saniye uyur: 7/8
Toplam geçen zaman = 66.993584 saniye
Ana thread sonlandı...
```

Adım 2 – multithread.c: Çok-threadli program derlenmiş ve pthread kütüphanesi ile çalıştırılmıştır. Programda ana thread ile iki child thread (func1 ve func2) paralel olarak çalışmaktadır. Toplam çalışma süresi yaklaşık 36 saniye sürmüştür.

multithread.c kodu bu şekilde iken:

```
GNU nano 7.2                                multithread.c
/* multithread.c */
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
#include <sys/time.h>
#include <pthread.h>

/* Fonk. Tanımları (func1.c/func2.c içinde wrapper'lar tanımlı) */
void *func1_thread(void *arg);
void *func2_thread(void *arg);

int r1 = 0, r2 = 0;

int main(void)
{
    pthread_t td1, td2;
    int p;
    int j;
    float delta;
    struct timeval time1, time2;

    printf("Çok-threadli process ana thread olarak başlıyor...\n");
    gettimeofday(&time1, 0);

    /* 2 child thread yaratılması */
    //p = pthread_create(&td1, NULL, func1_thread, NULL);
    //if (p != 0) { perror("Thread 1 creation problem"); exit(1); }

    //p = pthread_create(&td2, NULL, func2_thread, NULL);
    //if (p != 0) { perror("Thread 2 creation problem"); exit(1); }

    /* Şimdi ana thread bir şeyler yapıyor */
    for (j=1; j<=4; ++j)
    {
        printf("Ana thread çalışıyor: %d/4\n", j);
        sleep(3);
    }

    pthread_join(td1, NULL);
    pthread_join(td2, NULL);

    gettimeofday(&time2, 0);
    delta = (float)((1000000*time2.tv_sec + time2.tv_usec) -
                    (1000000*time1.tv_sec + time1.tv_usec))/1000000;

    printf("Toplam geçen zaman = %f saniye\n", delta);
    printf("Ana thread sonlandı...\n");
    return 0;
}
```

Multithread.c çıktısı ise bu şekilde olmuştur:

```
Çok-threadli process ana thread olarak başlıyor...  
Fonksiyon func1 yazar ve 4 saniye uyur: 1/10  
Ana thread çalışıyor: 1/4  
Fonksiyon func2 yazar ve 3 saniye uyur: 1/8  
Ana thread çalışıyor: 2/4  
Fonksiyon func2 yazar ve 3 saniye uyur: 2/8  
Fonksiyon func1 yazar ve 4 saniye uyur: 2/10  
Ana thread çalışıyor: 3/4  
Fonksiyon func2 yazar ve 3 saniye uyur: 3/8  
Fonksiyon func1 yazar ve 4 saniye uyur: 3/10  
Ana thread çalışıyor: 4/4  
Fonksiyon func2 yazar ve 3 saniye uyur: 4/8  
Fonksiyon func1 yazar ve 4 saniye uyur: 4/10  
Fonksiyon func2 yazar ve 3 saniye uyur: 5/8  
Fonksiyon func2 yazar ve 3 saniye uyur: 6/8  
Fonksiyon func1 yazar ve 4 saniye uyur: 5/10  
Fonksiyon func2 yazar ve 3 saniye uyur: 7/8  
Fonksiyon func1 yazar ve 4 saniye uyur: 6/10  
Fonksiyon func1 yazar ve 4 saniye uyur: 7/10  
Fonksiyon func1 yazar ve 4 saniye uyur: 8/10  
Fonksiyon func1 yazar ve 4 saniye uyur: 9/10  
Toplam geçen zaman = 36.005787 saniye  
Ana thread sonlandı...
```

Adım 3 – pthread_join kaldırılması: pthread_join satırları kaldırıldığında ana thread, child thread'leri beklemeden sonlandığı için toplam çalışma süresi azalmıştır. Bu, thread'lerin tamamlanmadan sonlanmasından kaynaklanmaktadır.(Bu bir kod iyileştirmesi değildir, deney amaçlıdır.)

```
Çok-threadli process ana thread olarak başlıyor..  
Ana thread çalışıyor: 1/4  
Ana thread çalışıyor: 2/4  
Ana thread çalışıyor: 3/4  
Ana thread çalışıyor: 4/4  
Toplam geçen zaman = 12.002162 saniye  
Ana thread sonlandı...
```

Adım 4 – multiprocessing.c: Thread'li yapı process (fork) yapısı ile yeniden oluşturulmuştur. Parent process func1, child process ise func2 fonksiyonlarını çalıştırmıştır. Toplam süre çok-threadli versiyonla benzerdir (~36 saniye). Ancak process başlatma ve yönetim maliyeti daha yüksektir.

multiprocess.c kodları bu şekilde iken:

```
GNU nano 7.2 multiprocess.c
/* multiprocess.c */
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
#include <sys/time.h>
#include <sys/types.h>
#include <sys/wait.h>

/* func1() ve func2() doğrudan func1.c/func2.c'de olacak */
void func1(void);
void func2(void);

int main(void)
{
    pid_t pid;
    float delta;
    struct timeval time1, time2;

    printf("Multiprocess (fork) programı başlıyor...\n");
    gettimeofday(&time1, 0);

    pid = fork();
    if (pid < 0) { perror("fork failed"); exit(1); }

    if (pid == 0) {
        /* child process: örneğin func2'yi yap */
        printf("Child process: func2 çalışıyor (pid=%d)\n", getpid());
        func2();
        printf("Child bitti\n");
        exit(0);
    } else {
        /* parent process: func1 yap */
        printf("Parent process: func1 çalışıyor (pid=%d)\n", getpid());
        func1();

        /* parent işini bitirdikten sonra child için bekle */
        waitpid(pid, NULL, 0);

        gettimeofday(&time2, 0);
        delta = (float)((1000000*time2.tv_sec + time2.tv_usec) -
                        (1000000*time1.tv_sec + time1.tv_usec))/1000000;

        printf("Toplam geçen zaman (multiprocess) = %f saniye\n", delta);
        return 0;
    }
}
```

Multiprocess.c çıktısı ise bu şekildedir:

```
bartu_sanli@bartu-computer:~/Lab3a$ ./multiprocess
Multiprocess (fork) programı başlıyor...
Parent process: func1 çalışıyor (pid=903)
Fonksiyon func1 yazar ve 4 saniye uyur: 1/10
Child process: func2 çalışıyor (pid=904)
Fonksiyon func2 yazar ve 3 saniye uyur: 1/8
Fonksiyon func2 yazar ve 3 saniye uyur: 2/8
Fonksiyon func1 yazar ve 4 saniye uyur: 2/10
Fonksiyon func2 yazar ve 3 saniye uyur: 3/8
Fonksiyon func1 yazar ve 4 saniye uyur: 3/10
Fonksiyon func2 yazar ve 3 saniye uyur: 4/8
Fonksiyon func2 yazar ve 3 saniye uyur: 5/8
Fonksiyon func1 yazar ve 4 saniye uyur: 4/10
Fonksiyon func2 yazar ve 3 saniye uyur: 6/8
Fonksiyon func1 yazar ve 4 saniye uyur: 5/10
Fonksiyon func2 yazar ve 3 saniye uyur: 7/8
Fonksiyon func1 yazar ve 4 saniye uyur: 6/10
Child bitti
Fonksiyon func1 yazar ve 4 saniye uyur: 7/10
Fonksiyon func1 yazar ve 4 saniye uyur: 8/10
Fonksiyon func1 yazar ve 4 saniye uyur: 9/10
Toplam geçen zaman (multiprocess) = 35.224361 saniye
```

Adım 5 – multithreadparam.c: Global değişkenlerin adresleri parametre olarak geçirilmiş ve her thread'in r1 ve r2 değerlerini artırması sağlanmıştır.

multithreadparam.c kodu bu şekilde iken:


```
/* multithreadparam.c (kilitli - yarış koşulu örneği) */
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
#include <sys/time.h>
#include <pthread.h>

int r1 = 0, r2 = 0;

struct pair {
    int *r1p;
    int *r2p;
};

void *worker(void *arg)
{
    struct pair *p = (struct pair *)arg;
    /* sleepen önce r1'i bir arttır */
    (*(p->r1p))++;

    /* sleep ile simüle et */
    sleep(1);

    /* sleepen sonra r2'yi bir arttır */
    (*(p->r2p))++;

    return NULL;
}

int main(void)
{
    pthread_t td1, td2;
    struct pair p;
    p.r1p = &r1;
    p.r2p = &r2;

    /* ana thread de r1/r2'yi arttırsın (direkt) */
}
```

```

/* ana thread de r1/r2'yi arttırsın (direkt) */
r1++; /* ana thread sleepten önce bir kez arttır */
/* burada istersen sleep yap */
sleep(1);
r2++; /* ana thread sleepten sonra bir kez arttır */

/* iki child thread yarat */
pthread_create(&td1, NULL, worker, &p);
pthread_create(&td2, NULL, worker, &p);

pthread_join(td1, NULL);
pthread_join(td2, NULL);

printf("r1 = %d, r2 = %d\n", r1, r2);

return 0;
}

```

Multithreadparam.c çıktısı ise bu şekildedir:

```

bartu_sanli@bartu-computer:~/Lab3a$ ./multithreadparam
r1 = 3, r2 = 3
bartu_sanli@bartu-computer:~/Lab3a$ ./multithreadparam
r1 = 3, r2 = 3
bartu_sanli@bartu-computer:~/Lab3a$ ./multithreadparam
r1 = 3, r2 = 3
bartu_sanli@bartu-computer:~/Lab3a$ |

```

3. Sonuç ve Değerlendirme

Bu deneyde thread'lerin nasıl oluşturulduğu, nasıl senkronize edildiği ve process'lerden farkları öğrenilmiştir. pthread_create, pthread_join gibi temel fonksiyonlar kullanılarak çok-threadli programlama mantığı uygulamalı olarak anlaşılmıştır. Ayrıca senkronizasyon eksikliğinde race condition oluştuğu gözlemlenmiştir.